Program Assembly and Testing

Software Development:

In general Software Development consists of **five** steps:

1) **Design:** The determination of the overall structure of the program and data from the functional specification.
   The determination of the Algorithm to implement the necessary functions.
2) **Coding:** The actual writing of instruction in a specific programming language to implement the Algorithm of the previous steps.
3) **Translation:** Creation of the patterns of 0s and 1s (the Object Code) from the program (Source Program) created in step 2.
4) **Testing:** The determination of whether the object code, when executed, actually causes the system to implement the intended function or not.
5) **Debugging:** The process of determining the source of failure (if any) found during testing, In order to eliminate them. Correction of a failure require a repetition of steps starting with the first or the second one.

Program can be written in any one of the three languages:

a) Machine Language b) Assembly Language or c) High Level Language

Assembly Language:

Program written in Assembly Language or a High Level Language must be translated to machine language for execution.

(After an Application Program is written in Assembly Language, it must be translated – Assembled - into Machine Code. Machine or Binary Code consists of 0s and 1s and is, ofcourse, the only language that can be interpreted by Microprocessor's Instruction Decoder. A special program called an Assembler is necessary for efficient translation)

Assembly Language Uses Mnemonic Representation for Operation Code, Data and Addresses. Advantages of Mnemonic or Symbolic representation:

- Help in remembering the function of instruction, address or data.
- Make writing and modification of the program easier.
- Use of symbolic name for addresses facilitate the insertion and deletion of instruction.

Assembler:

Accept source program written in a specific Assembly Language and generate object code for the corresponding microprocessor.

Cross Assembler: Run on one microprocessor but assemble program written for another microprocessor.

Self Assembler: An assembler that run on a Microprocessor and create object code for the same microprocessor is called a self-assembler.

Assembler Source Program: Three type of statements are common in an Assembly Language Source program:

1) Microprocessor Instructions: That are to be translated into Machine Language instruction, resulting in object code.
2) Directive Statements or Pseudo Instruction: To the assembler program – directing the translati9on of the symbolic microprocessor instructions.
3) Comment Statements: Are for documentation purpose.

Most Assembler instruction consists of four separate fields:

1) Label Field: Name to be assigned to a instruction location.
2) Operation Code Field: Specifying the o0epration to be performed.
3) Operand Field: Providing Address and / or Data Information when required by op-code.
4) Comments: Describing the way in which the instruction related to the purpose of program.
   These fields are separated by following delimiters:
   : (Colon) After Each Label
   (Space) between Op-code and operand.
   , (Comma) between operands in an operand field.
   ; (Semi-Colon) preceding Comments.

Assembler Directives – Pseudo Instructions:

Pseudo Instructions are commands placed in a program to provide information to the assembler.
They are not part of the instruction set of the microprocessor, nor are they translated into executable code.
Some Assembler directive of Intel's ASM80 are:

1) The Origion; **ORG**, pseudo Instruction tells the assembler the location in memory for which the next instruction or data byte should be assembled.
   ORG expression (16-bit address)
2) End Assembly; **End**, pseudo directive must be the last instruction to explicitly indicate the end of the program.
   END
3) **Equate** or **SET** Assembler directive: Data constants are defined using equate or SET assembler directive. Equate assembler directive usually appears as a group at the beginning of the program.

name	EQU	expression
symbolic name		value / constant
COUNT	EQU	0100H

SET directive also assign a value to the name associated, however, the same symbol can be redefined at various points using SET.

name	SET	expression

4) Define Storage (**DS**): reserves or allocate read/write memory locations (of specified size), for storage of temporary data. The Label referred to the first of the location allocated.
   optional_label: DS expression (size)
   example
   TEMP1: DS 1
   TEMP2: DS 1
   creates two 1-byte storage registers in R/W memory with the names TEMP1 and TEMP2.
   Instructions in the program can read or write these memory locations, using memory reference instruction such as:
   STA TEMP1
   LDA TEMP2
   Similarly, memory buffers (Collection of consecutive memory locations) can also be created using Define storage.
   BUFF: DS 80
5) Define Byte (**DB**) places a fixed data value after allocating memory.
   optional_label: DB list
   TABLE: DB 00, 01, 03, 07, 0F, FF
   'list' refer to either to one or more 8-bit data quantities or string of character enclosed in quotes.

6) Define Word (**DW**):
   optional_label: DW list
   'list' refer to either to one or more 16-bit data quantities stored as 2 bytes.
7) Conditional Assembly Pseudo Instruction:
   IF expression
   [Assembly Language Statements]
   ENDIF

The assembler evaluates the expression, if it is 0, the statements in between IF and ENDIF are ignored, if it is 1, these statements are assembled.

8) Local Pseudo Directive:
   Local label_names

This directive is used in MACRO blocks. The specified label names are defined to have memory only within the current macro expansion. Each time the macro is reference and expanded, the assembler assigns each local symbol a unique symbol in the form ??nnnn.

MACROS

- A MACRO (instruction) is a collection of many instructions. It is a single instruction that the MACRO assembler replaces with a group of insertions whenever it appears in an Assembly Language Program.
- The MACRO and the instructions that are replaced by the MACRO instruction are defined only once by the program.
- MACROs are useful when a small group of instruction is repeated several times in a program.
- The following 3 steps are involved in a MACRO in an assembly language program.
  1) The MACRO definition: It defines the group of instructions equivalent to the MACRO instruction.

Label	Code	Operand
name	MACRO	list
	[Macro Body]	
	ENDM	
Name of macro		list of dummy parameters

  2) The MACRO reference: It is the use of the MACRO (instruction) as an instruction in the program.

  3) MACRO Expansion: It is the replacement of MACRO references with the group of instructions defined in MACRO definition, performed by the assembler, whenever it encounters a macro reference.

Example:

LHLD addr	(L)	<-	((Byte 3)(Byte 2))
	(H)	<-	((Byte 3)(Byte 2) +1)
MOV r, M	(r)	<-	((H)(L))

  1) Example Defining a MACRO

```
LDIND   MACRO   REG, ADDR
LHLD    ADDR
MOV     REG,M
ENDM
```

  2) MACRO Reference

```
LDIND   C, PTR
```

3) MACRO Expansion

```
LHLD    PTR
MOV     C,M
```

Two Pass Assembler:

One pass one, a symbol table is generated, that defines the value of all symbols, label, and symbol assigned values by pseudo instruction.

The Assembler contains an Operation Code Table that has an entry for each op-code in the instruction set. Each table entry contains instruction mnemonics, its machine code equivalent and a descriptor word. The descriptor word indicates the number of bytes in the instruction and the format of the data in the operands. The assembler maintains Location Counter, LC, in memory, which is initially set to 0.

Process:

Pass 1:

- Each Line of the program is scanned.
- If the line has a label, that label is added to the symbol table along with the value of location counter, which defines its value.
- The descriptor word of the operation code is checked in operation code table and the location counter is incremented by the value specified for it in its descriptor word.
- In this way the location counter keeps track of the memory location assigned to each instruction.
- If the program contains an Origion pseudo instruction, the location counter is set to the value specified by the operand of the ORG pseudo instructions.
- If an attempt is made to set the location counter to a value less than its present value, an error occurs.
- When the last line of the program is scanned, (END) the first pass of the assembly is complete.

Pass 2:

- The second pass of a two-pass assembler scans the program again.
- Using the operation code table and the symbol table created in the first pass, it replaces all the symbols with their machine code equivalent and thus generates the binary object code.

******